

Chapter 5: Design Consistent Hashing

Introduction

This chapter explores consistent hashing, a technique essential for achieving horizontal scaling by efficiently distributing requests and data across servers. It minimizes data redistribution when servers are added or removed and ensures an even distribution of data to mitigate issues like server hotspots.

The Rehashing Problem

Explanation

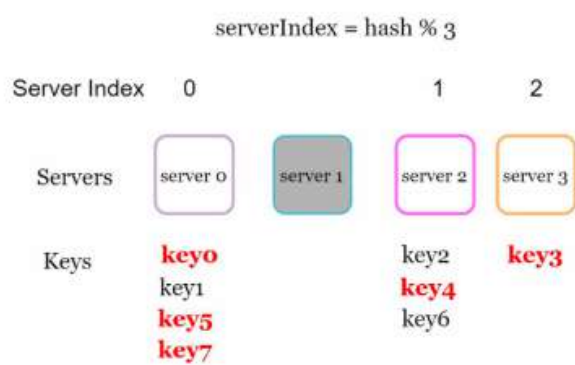
In traditional hashing methods, such as `serverIndex = hash(key) % N`, data redistribution becomes problematic when the number of servers changes. For example:

- Removing a server causes most keys to be reassigned, leading to cache misses.
- Adding a server results in unnecessary key redistributions.

serverIndex = hash % 4

Server Index	0	1	2	3
Servers	server 0	server 1	server 2	server 3
Keys	key1 key3	key0 key4	key2 key6	key5 key7

- This approach works well when the size of the server pool is fixed. However, problems arise when new servers are added, or existing servers are removed.



Key Issue

Redistribution of most keys when server count changes causes inefficiency and overload.

Consistent Hashing

Definition

Consistent hashing ensures that only a fraction of keys are remapped when servers are added or removed. This minimizes disruptions and enhances scalability.

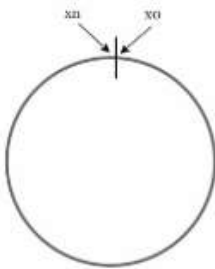
Key Concepts

- 1. **Hash Space and Ring:** The hash space forms a continuous ring, with hash values distributed from 0 to $2^{160}-1$ (e.g., using hash function like SHA-1). By connecting both ends we get a ring.

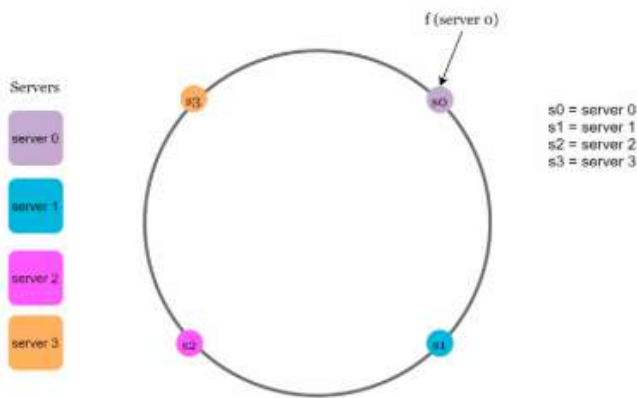


Figure 5-3

By collecting both ends, we get a hash ring as shown in Figure 5-4:



- Using the same hash function f , we map servers based on server IP or name onto the ring.



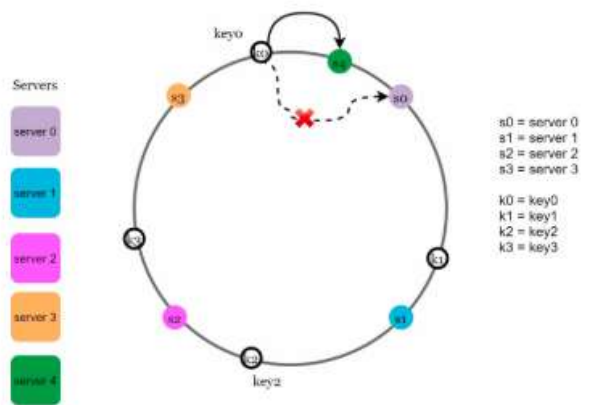
1. Server Lookup

- A key's server is determined by traversing clockwise on the ring until a server is found.

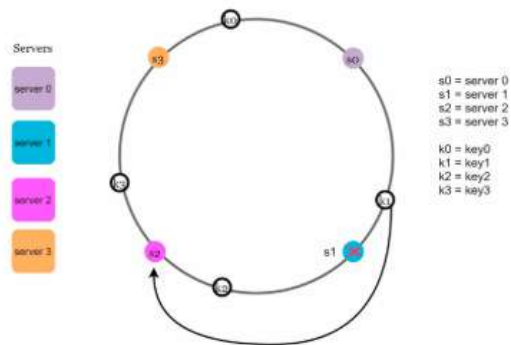


2. Adding and Removing Servers

- Adding a server redistributes only nearby keys. Only a fraction of keys are redistributed to the new server.



- Removing a server affects only the keys in its range. Only keys from the removed server are reassigned to the next server clockwise.



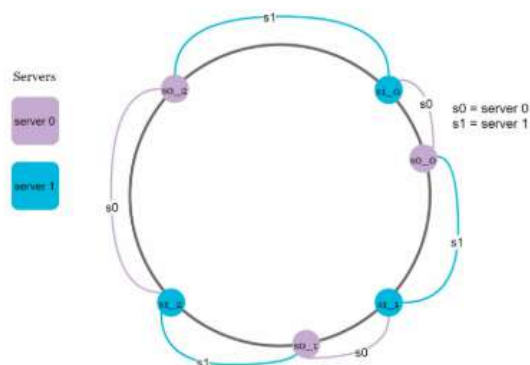
Challenges and Solutions

Two Issues in Basic Approach

1. **Uneven Partition Sizes:** Servers may have unequal data partitions.
2. **Non-uniform Key Distribution:** Some servers may receive significantly more keys than others.

Solution: Virtual Nodes

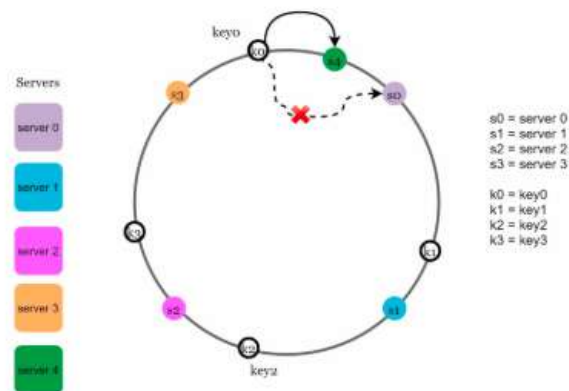
- Each server is represented by multiple virtual nodes on the ring uniformly distributed on the ring.
- Virtual nodes improve key distribution and balance load. As the number of virtual nodes increases, the distribution of keys becomes more balanced. This is because the standard deviation gets smaller with more virtual nodes, leading to balanced data distribution.



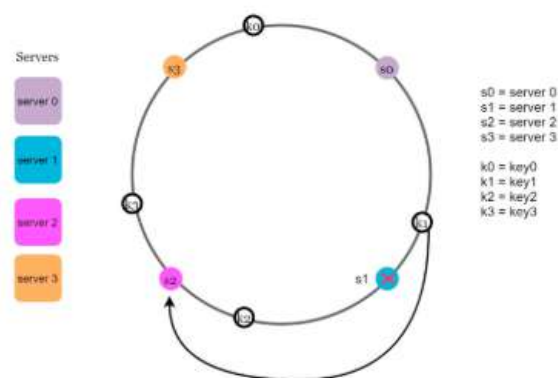
Affected Keys

When servers are added or removed:

- **Added Server:** Affected keys are those between the new server and its predecessor. In the following example server 4 is added onto the ring. The affected range starts from s_4 (newly added node) and moves anticlockwise around the ring until a server is found (s_3). Thus, keys located between s_3 and s_4 need to be redistributed to s_4 .



- **Removed Server:** Affected keys are those between the removed server and its predecessor. In the following example when a server (s_1) is removed, the affected range starts from s_1 (removed node) and moves anticlockwise around the ring until a server is found (s_0). Thus, keys located between s_0 and s_1 must be redistributed to s_2 .



🔗 Benefits of Consistent Hashing

- **Minimized Redistribution:** Only a fraction of keys are reassigned.
- **Scalability:** Enables horizontal scaling.
- **Mitigates Hotspots:** Balances data distribution to avoid server overload.

Real-World Applications

- Amazon Dynamo DB
- Apache Cassandra
- Discord
- Akamai CDN
- Maglev Load Balancer